

Informe unificado - Arquitectura y funcionamiento del FRONTEND

SoftwareReparacionEquipos

Versión basada en el repositorio SoftwareReparacionEquipos-master (frontend Vite/React) y el documento de referencia "Análisis_Arquitectonico_SoftwareReparacionEquipos.docx".

1. Resumen ejecutivo (frontend)

El frontend es una SPA en React (Vite) sin React Router. La navegación se implementa mediante estado local en AdminDashboard.jsx: primero decide por autenticación (token), luego por rol (Administrador/Recepción/Técnico/Cliente) y, únicamente para Administrador, por una vista interna ('view') que alterna secciones (Overview, Órdenes, Clientes, etc.). El componente AdminDashboard concentra también el "store" global de datos (arrays en memoria), el cliente API (helpers apiGet/apiPost/apiPut/apiDelete), los filtros (useMemo) y el control de modales.

Hechos base documentados: estructura de archivos y routing por estado, ausencia de React Router, y secciones del Admin.

Referencia: documento de arquitectura (sección 4.1 y 4.2).

2. Estructura del frontend (carpetas y archivos)

Estructura declarada en el documento de referencia:

```
frontend/  
  index.html  
  vite.config.js  
  package.json  
  src/  
    main.jsx  
    App.jsx  
    App.css / index.css  
    auth/views/LoginView.jsx  
    admin/views/AdminDashboard.jsx  
    admin/views/sections/*  
    admin/components/AdminFormsAndModals.jsx  
    tecnico/views/TecnicoDashboard.jsx  
    recepcion/views/RecepcionDashboard.jsx  
    cliente/views/ClienteDashboard.jsx
```

Notas:

- App.jsx renderiza exclusivamente <AdminDashboard/>.
- AdminDashboard actúa como router por rol, store global y controlador de modales.
- AdminFormsAndModals.jsx concentra formularios y modales (18 componentes reutilizables).

3. Navegación (routing) en el frontend: autenticación, rol y vista

3.1. No hay React Router: routing por estado

No existe ruteo por URL (no hay rutas /admin, /ordenes, etc.). La aplicación opera como un panel tipo "single page" donde el componente raíz elige qué renderizar según estado local.

3.2. Flujo de decisión de render (pseudosecuencia)

```
if (!token) {
  render <LoginView/>
} else {
  switch(user.rol) {
    case 'Tecnico'/'Técnico': render <TecnicoDashboard/>
    case 'Recepcion':        render <RecepcionDashboard/>
    case 'Cliente':          render <ClienteDashboard/>
    case 'Administrador':    render AdminLayout(view)
  }
}
```

Este flujo está descrito explícitamente en el documento de referencia (sección 4.2) y se implementa en AdminDashboard.jsx. La “navegación” del Administrador se controla con el estado view y render condicional de secciones.

4. Archivo por archivo: roles y flujo funcional (detallado)

4.1. main.jsx (punto de entrada)

Responsabilidad: montar React en #root y renderizar <App/>.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

4.2. App.jsx (raíz de la SPA)

4.3. LoginView.jsx (landing + login + registro de cliente)

Responsabilidad: UI pública de autenticación. Incluye:

- Formulario de login (email/password).
- Registro de cliente (crea Cliente + Usuario con rol Cliente).
- Manejo de validaciones y mensajes de error a nivel UI.

Fragmento relevante: flujo de registro de cliente (endpoint /api/auth/register-cliente).

```

      Perfiles disponibles: Administrador, Técnico, Recepción y Cliente.
    </div>
    <button type="button" className="link-button" onClick={() =>
setShowRegister((v) => !v)}>
      {showRegister ? 'Ocultar registro cliente' : '¿Eres cliente nuevo? Crea
tu cuenta'}
    </button>
  </form>

  {showRegister ? (
    <form className="login-form register-form" onSubmit={handleRegisterSubmit}>
      <h3>Registro de cliente</h3>
      <div className="filters">
        <input placeholder="Nombre" value={regForm.nombre} onChange={(e) =>
handleRegChange('nombre', e.target.value)} />
        <input placeholder="Apellido" value={regForm.apellido} onChange={(e) =>
handleRegChange('apellido', e.target.value)} />
        <input placeholder="Documento" value={regForm.documento} onChange={(e)
=> handleRegChange('documento', e.target.value)} />
        <input placeholder="Teléfono" value={regForm.telefono} onChange={(e) =>
handleRegChange('telefono', e.target.value)} />
        <input placeholder="Email" type="email" value={regForm.email}
onChange={(e) => handleRegChange('email', e.target.value)} />

```

```

        <input placeholder="Ciudad" value={regForm.ciudad} onChange={(e) =>
handleRegChange('ciudad', e.target.value)} />
        <input placeholder="Dirección" value={regForm.direccion} onChange={(e)
=> handleRegChange('direccion', e.target.value)} />
        <input placeholder="Contraseña" type="password"
value={regForm.password} onChange={(e) => handleRegChange('password', e.target.value)}
/>
        <input placeholder="Confirmar contraseña" type="password"
value={regForm.confirm} onChange={(e) => handleRegChange('confirm', e.target.value)} />
    </div>
    {regError ? <div className="form-alert form-alert-error">{regError}</div>
: null}
    {regSuccess ? <div className="form-alert form-alert-
success">{regSuccess}</div> : null}
    <button type="submit" className="primary-button" disabled={regLoading}>
    {regLoading ? 'Creando cuenta...' : 'Crear cuenta cliente'}
    </button>

```

4.4. AdminDashboard.jsx (router por rol + store global + cliente API + modales)

Este archivo es el núcleo del frontend (948 líneas en el ZIP). Centraliza cinco capas:

- 1) Autenticación (token/user) y persistencia en localStorage.
- 2) Cliente API (apiGet/apiPost/apiPut/apiDelete) con header Authorization: Bearer <token>.
- 3) Store global: arrays (órdenes, clientes, equipos, técnicos, grupos, reglas, pagos, usuarios, roles, estados, tipos).
- 4) Filtros/búsqueda en cliente (useMemo).
- 5) Control de modales y handlers de mutación (crear/editar/eliminar + refresh).

Bloque	Descripción	Líneas (AdminDashboard.jsx)
Inicialización token/user desde localStorage	Verificación en el código del ZIP.	38, 40
Login exitoso guarda token/user	Verificación en el código del ZIP.	107, 108
Logout limpia localStorage	Verificación en el código del ZIP.	121, 122
Helper apiGet (inicio)	Verificación en el código del ZIP.	139
Carga paralela Promise.all de datasets base	Verificación en el código del ZIP.	193, 199
Branch de carga extra por rol (admin/recepción)	Verificación en el código del ZIP.	202, 205
Funciones refresh* (ordenes/clientes/...)	Verificación en el código del ZIP.	355, 363
Routing por rol (isTecnico, etc.)	Verificación en el código del ZIP.	547, 551, 569, 630

Nota: los números de línea corresponden al archivo del ZIP subido (AdminDashboard.jsx).

4.4.1 Autenticación y persistencia

Estados base:

- token: se inicializa leyendo localStorage('token').
 - user: se inicializa parseando localStorage('user').
- Si el parse falla, user queda null y se fuerza pantalla de login.

```

} from '../components/AdminFormsAndModals'

function AdminDashboard() {

```

```

const [token, setToken] = useState(() => localStorage.getItem('token') || '')
const [user, setUser] = useState(() => {
  const raw = localStorage.getItem('user')
  return raw ? JSON.parse(raw) : null
})

const [email, setEmail] = useState('')
const [password, setPassword] = useState('')
const [loading, setLoading] = useState(false)
const [error, setError] = useState('')
const [success, setSuccess] = useState('')
const [view, setView] = useState('overview')

const [ordenes, setOrdenes] = useState([])
const [clientes, setClientes] = useState([])
const [equipos, setEquipos] = useState([])
const [tecnicos, setTecnicos] = useState([])
const [grupos, setGrupos] = useState([])
const [reglas, setReglas] = useState([])
const [pagos, setPagos] = useState([])
const [usuarios, setUsuarios] = useState([])
const [roles, setRoles] = useState([])
const [estados, setEstados] = useState([])

```

4.4.2 Cliente API (apiGet/apiPost/apiPut/apiDelete)

Estos helpers encapsulan fetch() y normalizan:

- URL base: http://localhost:3001 (hardcode).
- Header Authorization: Bearer <token>.
- Content-Type JSON para POST/PUT.
- Manejo de error: si res.ok == false, lanza Error para que UI muestre banner/mensaje.

```

    if (!res.ok) throw new Error(data?.error || 'No se pudo crear la cuenta.')
    setEmail(payload.email || '')
    setSuccess('Cuenta cliente creada. Inicia sesión con tu email y contraseña.')
  }
}

const apiGet = async (path) => {
  const res = await fetch(`http://localhost:3001${path}`, {
    headers: { Authorization: `Bearer ${token}` },
  })
  if (!res.ok) throw new Error('Error al cargar datos')
  return res.json()
}

const apiPost = async (path, payload) => {
  const res = await fetch(`http://localhost:3001${path}`, {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(payload),
  })
  const data = await res.json()
  if (!res.ok) throw new Error(data?.error || 'Error al guardar')
  return data
}

const apiPut = async (path, payload) => {
  const res = await fetch(`http://localhost:3001${path}`, {
    method: 'PUT',
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(payload),
  })
  const data = await res.json()
  if (!res.ok) throw new Error(data?.error || 'Error al actualizar')
  return data
}

```

```

}

const apiDelete = async (path) => {
  const res = await fetch(`http://localhost:3001${path}`, {
    method: 'DELETE',
    headers: { Authorization: `Bearer ${token}` },
  })
  const data = await res.json()
  if (!res.ok) throw new Error(data?.error || 'Error al eliminar')
  return data
}

useEffect(() => {

```

4.4.3 Carga inicial y control por rol

useEffect dispara carga cuando hay token. Se construye un conjunto base (órdenes, clientes, equipos, técnicos, grupos, estados, tipos) y un conjunto extra condicionado por rol:

- Administrador: reglas + pagos + usuarios + roles.
- Recepción: pagos.
- Técnico/Cliente: solo base (según implementación).

```

if (!token) return
const load = async () => {
  try {
    setDataLoading(true)
    const isAdmin = user?.rol === 'Administrador'
    const isRecepcion = user?.rol === 'Recepcion' || user?.rol === 'Recepción'
    const basePromises = [
      apiGet('/api/ordenes'),
      apiGet('/api/clientes'),
      apiGet('/api/equipos'),
      apiGet('/api/tecnicos'),
      apiGet('/api/grupos'),
      apiGet('/api/estados'),
      apiGet('/api/tipos-equipo'),
    ]
    const extraPromises = isAdmin
      ? [apiGet('/api/reglas-asignacion'), apiGet('/api/pagos'),
        apiGet('/api/usuarios'), apiGet('/api/usuarios/roles')]
      : isRecepcion
      ? [Promise.resolve([]), apiGet('/api/pagos'), Promise.resolve([]),
        Promise.resolve([])]
      : [Promise.resolve([]), Promise.resolve([]), Promise.resolve([]),
        Promise.resolve([])]
    const [
      ordenesData,
      clientesData,
      equiposData,
      tecnicosData,
      gruposData,
      estadosData,
      tiposData,
      reglasData,
      pagosData,
      usuariosData,
      rolesData,
    ] = await Promise.all([...basePromises, ...extraPromises])
    setOrdenes(ordenesData)
    setClientes(clientesData)
    setEquipos(equiposData)
    setTecnicos(tecnicosData)
    setGrupos(gruposData)
    setEstados(estadosData)
    setTipos(tiposData)
    setReglas(reglasData)
    setPagos(pagosData)
    setUsuarios(usuariosData)
    setRoles(rolesData)
  } catch (err) {
    setError('No se pudieron cargar los datos del dashboard.')
  }
}

```

```

    } finally {
      setDataLoading(false)
    }
  }
}

```

4.4.4 Filtros y derivación de listas (useMemo)

Las listas se filtran en cliente (no hay endpoints de búsqueda). Esto se implementa con useMemo para evitar recalcular en cada render. En particular:

- Órdenes: texto (código/cliente/marca/modelo/serie), estado, técnico, grupo, y checkbox 'solo pendientes'.
- Clientes/Equipos: texto.
- Pagos: texto, método y rango de fechas.

```

import { useEffect, useMemo, useState } from 'react'
import '../App.css'
import LoginView from '../auth/views/LoginView'
import OverviewSection from './sections/OverviewSection'
import OrdenesSection from './sections/OrdenesSection'
import ClientesSection from './sections/ClientesSection'
import EquiposSection from './sections/EquiposSection'
import TecnicosSection from './sections/TecnicosSection'
import GruposSection from './sections/GruposSection'
import ReglasSection from './sections/ReglasSection'
import PagosSection from './sections/PagosSection'
import UsuariosSection from './sections/UsuariosSection'
import TecnicoDashboard from '../tecnico/views/TecnicoDashboard'
import RecepcionDashboard from '../recepcion/views/RecepcionDashboard'
import ClienteDashboard from '../cliente/views/ClienteDashboard'
import {
  GrupoForm,
  TecnicoForm,
  ClienteForm,
  EquipoForm,
  GrupoAssignForm,
  OrdenModal,
  OrdenCreateForm,
  ClienteModal,
  EquipoModal,
  TecnicoModal,
  OrdenDetalleModal,
  GrupoModal,
  GrupoTecnicosModal,
  ReglaForm,

```

4.4.5 Mutaciones: CRUD + refresh

Cada operación de escritura sigue el patrón:

- 1) apiPost/apiPut/apiDelete
- 2) refreshX (recarga la colección)
- 3) cierre de modal / limpieza de estado de edición

```

const csv = [header, ...lines]
  .map((row) => row.map((col) => `${String(col).replaceAll('"',
    '""')}`").join(','))
  .join('\n')
const blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' })
const url = URL.createObjectURL(blob)
const link = document.createElement('a')
link.href = url
link.setAttribute('download', `pagos_${new Date().toISOString().slice(0, 10)}.csv`)
document.body.appendChild(link)
link.click()
link.remove()
URL.revokeObjectURL(url)
}

const refreshOrdenes = async () => setOrdenes(await apiGet('/api/ordenes'))
const refreshClientes = async () => setClientes(await apiGet('/api/clientes'))
const refreshEquipos = async () => setEquipos(await apiGet('/api/equipos'))

```

```

const refreshTecnicos = async () => setTecnicos(await apiGet('/api/tecnicos'))
const refreshGrupos = async () => setGrupos(await apiGet('/api/grupos'))
const refreshReglas = async () => setReglas(await apiGet('/api/reglas-asignacion'))
const refreshPagos = async () => setPagos(await apiGet('/api/pagos'))
const refreshUsuarios = async () => setUsuarios(await apiGet('/api/usuarios'))
const refreshRoles = async () => setRoles(await apiGet('/api/usuarios/roles'))

const createOrden = async (payload) => {
  await apiPost('/api/ordenes', payload)
  await refreshOrdenes()
}

const updateOrdenEstado = async (orden, estadoId) => {
  await apiPut(`/api/ordenes/${orden.id}`, {
    codigo: orden.codigo,
    cliente_id: orden.cliente_id,
    equipo_id: orden.equipo_id,
    estado_id: estadoId,
    tecnico_id: orden.tecnico_id,
    grupo_id: orden.grupo_id,
    falla_reportada: orden.falla_reportada,
    diagnostico: orden.diagnostico,
    observaciones: orden.observaciones,
    costo_estimado: orden.costo_estimado,
    costo_final: orden.costo_final,
  })
  await refreshOrdenes()
}

const updateOrdenTecnico = async (orden, tecnicoId) => {
  await apiPut(`/api/ordenes/${orden.id}`, {
    codigo: orden.codigo,
    cliente_id: orden.cliente_id,
    equipo_id: orden.equipo_id,
    estado_id: orden.estado_id,
    tecnico_id: tecnicoId,
    grupo_id: orden.grupo_id,
    falla_reportada: orden.falla_reportada,
    diagnostico: orden.diagnostico,
    observaciones: orden.observaciones,
    costo_estimado: orden.costo_estimado,
    costo_final: orden.costo_final,
  })
  await refreshOrdenes()
}

const deleteOrden = async (ordenId) => {
  if (!confirm('¿Eliminar orden? (No se puede si tiene pagos)')) return
  try {
    await apiDelete(`/api/ordenes/${ordenId}`)
    await refreshOrdenes()
  } catch (err) {
    setError(err.message || 'No se pudo eliminar la orden.')
  }
}

```

4.4.6 Modales: control por estado

Los modales se activan manteniendo referencias de edición/detalle en estado (editOrden, viewOrden, editCliente, etc.). Si el estado no es null, el modal se renderiza como overlay. Al guardar, invoca callbacks y cierra.

```

const [roles, setRoles] = useState([])
const [estados, setEstados] = useState([])
const [tipos, setTipos] = useState([])
const [dataLoading, setDataLoading] = useState(false)
const [editOrden, setEditOrden] = useState(null)
const [editCliente, setEditCliente] = useState(null)
const [editEquipo, setEditEquipo] = useState(null)
const [editTecnico, setEditTecnico] = useState(null)
const [editRegla, setEditRegla] = useState(null)
const [editGrupo, setEditGrupo] = useState(null)
const [editUsuario, setEditUsuario] = useState(null)

```

```

const [resetPassUsuario, setResetPassUsuario] = useState(null)
const [viewGrupo, setViewGrupo] = useState(null)
const [viewOrden, setViewOrden] = useState(null)

const [ordenSearch, setOrdenSearch] = useState('')
const [ordenEstado, setOrdenEstado] = useState('')
const [ordenTecnico, setOrdenTecnico] = useState('')
const [ordenGrupo, setOrdenGrupo] = useState('')
const [ordenSoloPendientes, setOrdenSoloPendientes] = useState(false)

const [clienteSearch, setClienteSearch] = useState('')
const [equipoSearch, setEquipoSearch] = useState('')
const [pagoSearch, setPagoSearch] = useState('')
const [pagoMetodo, setPagoMetodo] = useState('')
const [pagoDesde, setPagoDesde] = useState('')
const [pagoHasta, setPagoHasta] = useState('')

```

4.4.7 Routing por rol y por view (Administrador)

Al final del componente, se calcula el rol y se retorna el dashboard correspondiente. Solo Administrador renderiza el layout con sidebar y navegación interna por 'view'.

```

        throw err
      }
    }
  }

  if (token && user) {
    const isAdmin = user.rol === 'Administrador'
    const isTecnico = user.rol === 'Tecnico' || user.rol === 'Técnico'
    const isRecepcion = user.rol === 'Recepcion' || user.rol === 'Recepción'
    const isCliente = user.rol === 'Cliente'

    if (isTecnico) {
      return (
        <TecnicoDashboard
          user={user}
          handleLogout={handleLogout}
          error={error}
          dataLoading={dataLoading}
          ordenes={ordenes}
          estados={estados}
          tecnicos={tecnicos}
          onQuickEstado={updateOrdenEstado}
          onSaveOrdenTrabajo={saveOrdenTrabajo}
        />
      )
    }

    if (isRecepcion) {
      return (
        <RecepcionDashboard
          user={user}
          handleLogout={handleLogout}
          error={error}
          dataLoading={dataLoading}
          ordenes={ordenes}
          clientes={clientes}
          equipos={equipos}
          estados={estados}
          tecnicos={tecnicos}
          grupos={grupos}
          tipos={tipos}
          onCreateOrden={createOrden}
          onCreateCliente={createCliente}
          onCreateEquipo={createEquipo}
          onUpdateOrdenEstado={updateOrdenEstado}
          onUpdateOrdenTecnico={updateOrdenTecnico}
          onSaveOrden={async (ordenId, payload) => {
            await apiPut(`/api/ordenes/${ordenId}`, payload)
            await refreshOrdenes()
          }}
        />
      )
    }
  }

```

```

    onSaveCliente={async (clienteId, payload) => {
      await apiPut(`/api/clientes/${clienteId}`, payload)
      await refreshClientes()
    }}
    onSaveEquipo={async (equipoId, payload) => {
      await apiPut(`/api/equipos/${equipoId}`, payload)
      await refreshEquipos()
    }}
    onRegisterPago={async (ordenId, payload) => {
      await apiPost(`/api/ordenes/${ordenId}/pagos`, payload)
      await Promise.all([refreshOrdenes(), refreshPagos()])
    }}
    pagos={pagos}
    onDownloadFactura={async (ordenId) => {
      const res = await
fetch(`http://localhost:3001/api/ordenes/${ordenId}/factura`, {
      headers: { Authorization: `Bearer ${token}` },
    })
      if (!res.ok) {
        const data = await res.json()
        throw new Error(data?.error || 'No se pudo descargar la factura')
      }
      const blob = await res.blob()
      const contentDisposition = res.headers.get('content-disposition') || ''
      const fileNameMatch = contentDisposition.match(/filename=\"?([^\\\"]+)\\"?/)
      const fileName = fileNameMatch?.[1] || `factura_orden_${ordenId}.pdf`
      const url = URL.createObjectURL(blob)
      const link = document.createElement('a')
      link.href = url
      link.download = fileName
      document.body.appendChild(link)
      link.click()
      link.remove()
      URL.revokeObjectURL(url)
    }}
  />
)
}

if (isCliente) {
  return (
    <ClienteDashboard
      user={user}
      token={token}
      handleLogout={handleLogout}
    />
  )
}

const navItems = [
  ['overview', 'Overview'],
  ['ordenes', 'Órdenes'],
  ['clientes', 'Clientes'],
  ['equipos', 'Equipos'],
  ['tecnicos', 'Técnicos'],
  ['grupos', 'Grupos'],
]

if (isAdmin) {
  navItems.push(['reglas', 'Reglas'])
  navItems.push(['pagos', 'Pagos'])
  navItems.push(['usuarios', 'Usuarios'])
}

return (
  <div className="dashboard">
    <aside className="sidebar">
      <div className="brand">
        <span className="brand-mark" aria-hidden="true">Admin</span>
        <div className="brand-text">
          <span className="brand-title">Reparacion De Equipos</span>
          <span className="brand-subtitle">Consola Administrador</span>
        </div>
      </div>
    </aside>
  </div>
)

```

```

</div>
<nav className="nav">
  {navItems.map(([key, label]) => (
    <button
      key={key}
      className={`nav-item ${view === key ? 'active' : ''}`}
      onClick={() => setView(key)}
    >
      {label}
    </button>
  ))}
</nav>
<div className="sidebar-footer">
  <div className="user-chip">
    <span>{user?.nombre} {user?.apellido}</span>
    <span className="user-role">{user?.rol}</span>
  </div>
  <button className="ghost-button" onClick={handleLogout}>Cerrar
sesión</button>
</div>
</aside>

<main className="dash-main">
  <header className="dash-header">
    <div>
      <h1>Panel de administración</h1>
      <p>Control general del taller y operaciones.</p>
    </div>
    <div className="header-actions">
      <span className="env-pill">Local</span>
    </div>
  </header>

```

4.5 Secciones del Administrador (admin/views/sections)

Estas secciones son componentes relativamente delgados: reciben listas ya filtradas y callbacks desde AdminDashboard. Su responsabilidad es: render tabla, capturar UI events (click), y delegar mutaciones al controlador.

Archivo	Responsabilidad UI	Tamaño (líneas)
OverviewSection.jsx	Métricas/resumen (contadores por estado).	29
OrdenesSection.jsx	Tabla de órdenes + filtros + acciones (ver, editar, cambiar estado, asignar, eliminar).	95
ClientesSection.jsx	Tabla de clientes + buscador + acciones CRUD.	44
EquiposSection.jsx	Tabla de equipos + buscador + acciones CRUD.	46
TecnicosSection.jsx	Tabla de técnicos + toggle activo + CRUD.	46
GruposSection.jsx	Tabla de grupos + asignación y visualización de técnicos del grupo.	51
ReglasSection.jsx	Gestión de reglas de asignación (solo admin).	35
PagosSection.jsx	Tabla de pagos + filtros + exportación CSV (según doc).	54

4.6 AdminFormsAndModals.jsx (formularios y modales reutilizables)

Este archivo concentra 18 componentes (formularios inline + modales). Su finalidad es estandarizar UI/validación y aislar la edición/creación en overlays.

Resumen de formularios (campos y validación) según documento de referencia:

Formulario	Propósito	Campos	Validación
GrupoForm	Crear entidad/relación	nombre, descripcion	nombre obligatorio
TecnicoForm	Crear entidad/relación	nombre, apellido, email, telefono, especialidad, activo	nombre y apellido obligatorios
ClienteForm	Crear entidad/relación	nombre, apellido, documento, telefono, email, ciudad, direccion	nombre y apellido obligatorios
EquipoForm	Crear entidad/relación	cliente(select), tipo(select), marca, modelo, serie, color, accesorios, descripcion	cliente, marca y serie obligatorios
OrdenCreateForm	Crear entidad/relación	codigo, cliente, equipo (filtrado), estado, grupo, tecnico, falla, costo estimado	codigo, cliente, equipo y falla obligatorios
ReglaForm	Crear entidad/relación	palabras clave, grupo, tecnico(opc), prioridad, activo	palabras clave y grupo obligatorios
UsuarioForm	Crear entidad/relación	nombre, apellido, email, telefono, rol, password, activo	todos excepto telefono; password >= 8
GrupoAssignForm	Crear entidad/relación	grupo(select), tecnico(select)	ambos seleccionados

Resumen de modales (propósito):

- OrdenModal: Editar orden completa (estado, técnico, grupo, costo final, falla, diagnóstico, observaciones).
- OrdenDetalleModal: Detalle solo lectura + carga de historial y pagos al montar.
- ClienteModal: Editar datos cliente.
- EquipoModal: Editar datos equipo.
- TecnicoModal: Editar datos técnico.
- GrupoModal: Editar grupo responsable.
- GrupoTecnicosModal: Ver técnicos asignados a un grupo (solo lectura).
- ReglaModal: Editar regla de asignación.
- UsuarioModal: Editar usuario.
- UsuarioPasswordModal: Reset de contraseña (min 8).

5. Dashboards por rol (detallado)

5.1 Técnico - TecnicoDashboard.jsx

Objetivo: flujo de trabajo técnico sobre órdenes asignadas. Restricción: no crea clientes/equipos/órdenes.

Características descritas: filtra por tecnico_id del usuario autenticado, búsqueda por código/marca/modelo, acciones rápidas de cambio de estado y modal de trabajo técnico.

Artefactos UI típicos dentro del dashboard técnico:

- Listado/tablas de órdenes asignadas (derivado de ordenes globales o cargadas para técnico).
- Dropdown para cambio rápido de estado (dispara PATCH /api/ordenes/:id/estado).
- Modal de trabajo (diagnóstico + observaciones técnicas + estado).

Fragmento relevante: filtrado de órdenes en el cliente.

```
const tecnicoMatches = useMemo(() => {
  const email = String(user?.email || '').toLowerCase()
  const nombre = String(user?.nombre || '').toLowerCase()
  const apellido = String(user?.apellido || '').toLowerCase()

  return tecnicos.filter((tecnico) => {
    const byEmail = email && String(tecnico.email || '').toLowerCase() === email
    const byName =
      nombre &&
      apellido &&
      String(tecnico.nombre || '').toLowerCase() === nombre &&
      String(tecnico.apellido || '').toLowerCase() === apellido
    return byEmail || byName
  })
}, [tecnicos, user])

const misTecnicoIds = useMemo(() => tecnicoMatches.map((tecnico) => tecnico.id),
[tecnicoMatches])

const misOrdenes = useMemo(() => {
  if (misTecnicoIds.length > 0) {
    return ordenes.filter((orden) => misTecnicoIds.includes(orden.tecnico_id))
  }
  return ordenes.filter((orden) => {
```

5.2 Recepción - RecepcionDashboard.jsx

Objetivo: operación completa del taller. Incluye pestañas Órdenes/Clientes/Equipos, creación encadenada (cliente -> equipo -> orden), edición de entidades existentes, cambio de estado/técnico, registro de pagos y descarga de facturas PDF.

- Pestañas: Ordenes, Clientes, Equipos (navegación interna local).
- Flujo encadenado: seleccionar Cliente y luego Equipo filtrado por cliente al crear orden.
- Registro de pagos (POST /api/ordenes/:id/pagos) y consulta/listado según permisos.
- Descarga de factura PDF (GET /api/ordenes/:id/factura para recepcion/admin).

Fragmento relevante: uso de OrdenCreateForm (creación de órdenes).

```
import { useMemo, useState } from 'react'

import {
  OrdenCreateForm,
  ClienteForm,
  EquipoForm,
  OrdenModal,
  ClienteModal,
  EquipoModal,
} from '../..//admin/components/AdminFormsAndModals'

function RecepcionPagoModal({ orden, onClose, onSave, loading }) {
```

```

const [metodo, setMetodo] = useState('Efectivo')
const [valor, setValor] = useState(String(Math.max(Number(orden.saldo || 0), 0)))
const [referencia, setReferencia] = useState('')
const [error, setError] = useState('')

const submit = async (event) => {
  event.preventDefault()
  setError('')
  const valueNum = Number(valor)

```

5.3 Cliente - ClienteDashboard.jsx

Objetivo: portal de autoservicio. Permite consultar mis órdenes, ver detalle (historial/pagos), editar perfil, pagar y descargar factura. Las rutas usadas suelen ser /api/mis-ordenes, /api/mis-ordenes/:id y /api/clientes/me.

- Carga inicial: mis órdenes + perfil del cliente (GET /api/mis-ordenes, GET /api/clientes/me).
- Detalle por orden: historial de estados y pagos.
- Pago: habilitado cuando hay costo_final y estado apropiado (regla de negocio).
- Factura: disponible cuando la orden está pagada.

Fragmento relevante: carga de mis órdenes (endpoint /api/mis-ordenes).

```

    if (!res.ok) throw new Error(data?.error || 'Error al actualizar')
    return data
  }

const loadOrdenes = async () => {
  setLoading(true)
  setError('')
  try {
    const rows = await apiGet('/api/mis-ordenes')
    setOrdenes(rows)
  } catch (err) {
    setError(err.message || 'No se pudieron cargar tus órdenes.')
  } finally {
    setLoading(false)
  }
}

const loadPerfil = async () => {
  try {
    const data = await apiGet('/api/clientes/me')
    setPerfil(data)
    setPerfilForm({
      nombre: data.nombre || '',
      apellido: data.apellido || '',
      telefono: data.telefono || '',
      direccion: data.direccion || '',

```

Fragmento relevante: carga/actualización de perfil (endpoint /api/clientes/me).

```

      setError(err.message || 'No se pudieron cargar tus órdenes.')
    } finally {
      setLoading(false)
    }
  }
}

const loadPerfil = async () => {
  try {
    const data = await apiGet('/api/clientes/me')
    setPerfil(data)
    setPerfilForm({
      nombre: data.nombre || '',
      apellido: data.apellido || '',
      telefono: data.telefono || '',
      direccion: data.direccion || '',
      ciudad: data.ciudad || '',
    })
  } catch (err) {

```

```
        setError(err.message || 'No se pudo cargar tu perfil.')
    }
}

useEffect(() => {
    loadOrdenes()
    loadPerfil()
}, [])
```

5.4 Administrador - Panel completo (AdminDashboard + sections)

- Objetivo: control total. Adicional a Recepción, el Administrador gestiona:
- Reglas de asignación (ReglasSection).
 - Usuarios y roles (UsuariosSection).
 - Vista global de pagos (PagosSection) y utilidades como exportación (según documento).
- Sidebar interno que cambia el estado view.
 - Secciones: Overview, Órdenes, Clientes, Equipos, Técnicos, Grupos, Reglas, Pagos, Usuarios.
 - Uso intensivo de modales para edición/detalle y reset de password.

6. Matriz Acción UI -> Endpoint (para rastrear impacto)

La siguiente matriz resume las acciones principales del frontend y el endpoint correspondiente. Los endpoints están documentados en el análisis de backend del documento de referencia.

Acción	UI (rol/archivo)	Endpoint
Login	LoginView / AdminDashboard	POST /api/auth/login
Registro cliente	LoginView	POST /api/auth/register-cliente
Listar órdenes	Admin/Recepción/Técnico	GET /api/ordenes
Crear orden	Admin/Recepción	POST /api/ordenes
Editar orden	Admin/Recepción	PUT /api/ordenes/:id
Cambiar estado	Admin/Recepción/Técnico	PATCH /api/ordenes/:id/estado
Asignar técnico	Admin/Recepción	POST /api/ordenes/:id/asignar
Registrar pago	Admin/Recepción/Cliente	POST /api/ordenes/:id/pagos
Factura (admin/recepción)	Admin/Recepción	GET /api/ordenes/:id/factura
Mis órdenes (cliente)	Cliente	GET /api/mis-ordenes
Perfil cliente (cliente)	Cliente	GET/PUT /api/clientes/me
Usuarios	Admin	GET/POST/PUT /api/usuarios; PUT /api/usuarios/:id/password
Reglas asignación	Admin	GET/POST/PUT /api/reglas-asignacion

7. Pendientes relevantes del frontend (calidad, despliegue, escalabilidad)

- URL base hardcodeada (`http://localhost:3001`): migrar a `import.meta.env.VITE_API_URL` para dev/prod.
- No hay React Router: no existe deep-linking ni back/forward real; si es requisito, migrar a `react-router-dom`.
- AdminDashboard es monolítico (948 líneas): separar en hooks (`useAuth/useCatalogos/useOrdenes`) o store (`Context/Zustand`).
- Filtrado 100% en cliente: con grandes volúmenes requerirá paginación/búsqueda del lado del backend.
- Manejo de errores: centralizar en un `ErrorBanner/Toast` para consistencia entre dashboards.

8. Referencias

R1) “Análisis_Arquitectonico_SoftwareReparacionEquipos.docx” (documento subido por el usuario):

- Sección 4.1: Estructura de archivos del frontend.
- Sección 4.2: Flujo de navegación (routing por estado) y ausencia de React Router.
- Sección 4.3 y 4.4: Descripción de AdminDashboard y AdminFormsAndModals.
- Sección 4.5: Dashboards por rol.
- Sección 3.3: Mapa de endpoints (para cruzar acciones UI con API).

R2) Código fuente del ZIP “SoftwareReparacionEquipos-master.zip” (frontend): Archivos principales: `src/main.jsx`, `src/App.jsx`, `src/auth/views/LoginView.jsx`, `src/admin/views/AdminDashboard.jsx`, `src/admin/components/AdminFormsAndModals.jsx`, `src/tecnico/views/TecnicoDashboard.jsx`, `src/recepcion/views/RecepcionDashboard.jsx`, `src/cliente/views/ClienteDashboard.jsx`.